

Metodología de Sistemas II

Trabajo Práctico 1: Calidad de Diseño y Construcciones

Farias, Gustavo
Matrícula: 101662
Comisión: M2025-13

Repositorio GitHub: <https://github.com/Lucenear/UTN-TUPaD-TPs/tree/main/>

Parte A - Análisis de Calidad

1. Identificación de Code Smells

Code Smell	Ubicación	Impacto en el Sistema
Strings manuales	<code>"OSDE"</code> , <code>"SWISS"</code> , <code>"PUBLICA"</code> en condicionales.	Si el nombre de una obra social cambia, se debe buscar y reemplazar en múltiples líneas con alto riesgo de error.
Codigo duplicado	<code>`System.out.println("Pacien te premium")`</code> repetido para OSDE y SWISS.	Cualquier cambio en el mensaje o lógica de "premium" requiere editar dos lugares distintos.
Muchos parametros	Método <code>`procesar(String, int, String, boolean)`</code> .	Dificulta la lectura, invita a errores de orden en los argumentos y complica futuras extensiones.
Tipos primitivos	Uso de <code>`String`</code> para Obra Social y <code>`boolean`</code> para Urgencia.	No hay validación de tipo en tiempo de compilación; cualquier string es aceptado como obra social válida.
Condional redundante	<code>`if (urgente == true)`</code>	Innecesario. Reduce legibilidad sin aportar valor semántico.
Metodo con mucha responsabilidad	El método <code>`procesar`</code> valida, clasifica, construye strings y gestiona la lista.	Cualquier cambio en validación, formato o almacenamiento obliga a modificar este único método.

2. Problemas de Mantenibilidad

- Acoplamiento a la Consola: La clase imprime directamente con `System.out.println` dentro de la lógica de negocio. Esto impide escribir pruebas unitarias automatizadas y acopla las reglas del dominio al mecanismo de salida, imposibilitando cambiar a un logger o interfaz web sin reescribir la lógica central.
- Validaciones Embebidas en Lógica de Negocio: Las verificaciones de nombre no vacío y edad positiva están anidadas dentro del flujo principal de procesamiento. Modificar una regla de validación implica navegar y

comprender toda la estructura condicional del método procesar, aumentando el riesgo de regresiones.

- Representación Frágil del Turno: El turno se construye mediante concatenación manual de cadenas (nombre + "-" + edad + ...). Cambiar el formato del turno requiere editar esta concatenación dentro de un método ya sobrecargado, sin un modelo que centralice y proteja esa responsabilidad.

3. Ejemplos de Deuda Técnica

- Ausencia de Modelo de Dominio: Durante la pandemia se priorizó la velocidad de entrega y se usaron tipos primitivos en lugar de clases como Paciente u ObraSocial. Esta decisión consciente generó deuda de diseño que hoy hace costoso agregar atributos (DNI, teléfono) o comportamientos al paciente, requiriendo cambios en múltiples firmas de métodos y construcciones de cadenas.
- Validaciones Incompletas por Urgencia Temporal: Solo se verifica edad > 0 y nombre no vacío. No se validan rangos razonables de edad ni formatos de nombre. Esta omisión deliberada para acelerar el desarrollo ahora permite datos inconsistentes en producción, generando retrabajo futuro en limpieza de datos y corrección de registros erróneos.

4. Impacto en el Costo del Cambio

Los problemas identificados generan un costo de cambio no lineal. Un cambio aparentemente simple como agregar una nueva obra social "PAMI" requiere:

- Agregar un nuevo bloque else if con otra cadena mágica.
- Decidir y duplicar/extender la lógica de clasificación ("premium" o "publico").
- Verificar manualmente que la construcción del string del turno funcione correctamente.
- Realizar pruebas manuales completas porque no existen pruebas automatizadas que validen el comportamiento.

Cada modificación toca el mismo método monolítico procesar, aumentando la probabilidad de introducir errores en funcionalidades existentes. El tiempo estimado para un cambio seguro crece exponencialmente respecto al tamaño real del cambio solicitado.

Parte B - Calidad Interna vs Calidad Externa

Qué aspectos representan calidad externa?

El sistema asigna turnos correctamente, distingue entre pacientes premium y públicos según su obra social, marca turnos urgentes cuando corresponde y permite listar todos los turnos registrados. Desde la perspectiva del usuario final del Hospital Central, el comportamiento observable es correcto y cumple con los requisitos funcionales establecidos durante la pandemia.

Qué aspectos representan calidad interna?

La estructura del código presenta alta complejidad ciclomática (tres niveles de anidamiento condicional), duplicación de lógica, nombres poco descriptivos (procesar, mostrar, turnos), acoplamiento fuerte a la consola, ausencia de abstracciones de dominio y violaciones a principios SOLID. Estos atributos son invisibles para el usuario pero determinan directamente la facilidad de mantenimiento y evolución del sistema.

Puede un sistema funcionar correctamente y aun así tener mala calidad?

Sí, y este código es un ejemplo concreto. Cumple perfectamente su función externa de gestión de turnos pero posee calidad interna deficiente. La funcionalidad correcta es condición necesaria pero no suficiente para la sostenibilidad del software a largo plazo. Un sistema puede ser funcionalmente perfecto hoy y técnicamente insostenible mañana.

Qué riesgos genera mantener deuda técnica a largo plazo?

- Incremento exponencial del costo de cambio: Cada nueva funcionalidad requiere proporcionalmente más esfuerzo que la anterior debido al acoplamiento y la falta de abstracciones.
- Parálisis del equipo de desarrollo: Los programadores evitan tocar el código por miedo a introducir errores no detectables, retrasando entregas y acumulando más deuda.
- Imposibilidad de testing automatizado: El acoplamiento a la consola y la falta de modelos de dominio impiden crear una red de seguridad de pruebas, haciendo cada despliegue un acto de fe.
- Degradación progresiva (Ley de Lehman): Sin refactorización continua, el sistema se vuelve incomprensible y eventualmente requiere una reescritura completa, cuyo costo es órdenes de magnitud mayor que el mantenimiento preventivo oportuno.

Parte C - Refactor Inicial: Explicación de Mejoras Realizadas

Mejora Aplicada	Codigo Original	Codigo Refactorizado	Justificación Técnica
Nombres descriptivos	<code>`procesar()`, `mostrar()`, `turnos`</code>	<code>`agendarTurno()`, `listarTurnos()`, `turnosRegistrados`</code>	Un nuevo desarrollador entiende el propósito sin leer la implementación completa.
Enumerado ObraSocial	<code>Strings "OSDE", "SWISS", "PUBLICA"</code>	<code>`enum ObraSocial { OSDE, SWISS, PUBLICA }`</code>	Elimina strings literales y centraliza la categoría en un solo lugar extensible.
Clase Paciente	4 parámetros primitivos dispersos	Objeto <code>`Paciente`</code> encapsulado	Reduce la lista de parámetros, agrupa datos relacionados y permite mover validaciones y comportamientos al propio objeto.
Eliminación de duplicación	<code>Print "premium"</code> repetido 2 veces	Método <code>`getCategoria()`</code> en el enum	Principio DRY. Agregar nueva obra social requiere solo agregar un valor al enumerado.
Reducción de complejidad	3 niveles de anidamiento + <code>`== true`</code>	Validación temprana + <code>`if (urgente)`</code>	Menor complejidad y mayor legibilidad
Separación de responsabilidades	Todo concentrado en un método	Validación en <code>`Paciente`</code> , formato en <code>`Turno`</code> , orquestación en <code>`TurnoManager`</code>	Cada clase tiene una única razón de cambio. Cambiar formato no toca validación ni persistencia.

Creacion y refactorización del código:

ObraSocial.java

Este enum centraliza la clasificación de pacientes.

```
public enum ObraSocial {
    OSDE("Paciente premium"),
    SWISS("Paciente premium"),
    PUBLICA("Paciente publico");

    private final String categoria;

    ObraSocial(String categoria) {
        this.categoria = categoria;
    }

    public String getCategoria() {
        return categoria;
    }
}
```

Paciente.java

Encapsula los datos y mueve la validación fuera del método principal.

```
public class Paciente {
    private final String nombre;
    private final int edad;
    private final ObraSocial obraSocial;
    private final boolean urgente;

    public Paciente(String nombre, int edad, ObraSocial
obraSocial, boolean urgente) {
        this.nombre = nombre;
        this.edad = edad;
        this.obraSocial = obraSocial;
        this.urgente = urgente;
    }

    public boolean esValido() {
        return nombre != null && !nombre.isEmpty() && edad > 0;
    }

    public String getNombre() { return nombre; }
}
```

```

    public int getEdad() { return edad; }
    public ObraSocial getObraSocial() { return obraSocial; }
    public boolean isUrgente() { return urgente; }
}

```

Turno.java

Centraliza la construccion del string del turno, eliminando la concatenacion dispersa.

```

public class Turno {
    private final Paciente paciente;

    public Turno(Paciente paciente) {
        this.paciente = paciente;
    }

    @Override
    public String toString() {
        String representacion = paciente.getNombre() + "-"
                                + paciente.getEdad() + "-"
                                + paciente.getObraSocial().name();
        if (paciente.isUrgente()) {
            representacion += "-URGENTE";
        }
        return representacion;
    }
}

```

TurnoManager.java (Refactorizado)

Orquesta el proceso con responsabilidad unica y sin duplicacion.

```

import java.util.ArrayList;
import java.util.List;

public class TurnoManager {
    private final List<Turno> turnosRegistrados = new
    ArrayList<>();

    public void agendarTurno(Paciente paciente) {
        if (!paciente.esValido()) {
            return;
        }
    }
}

```

```
System.out.println(paciente.getObraSocial().getCategoria());
```

```
    Turno turno = new Turno(paciente);  
    turnosRegistrados.add(turno);  
    System.out.println("Turno agregado");  
}
```

```
public void listarTurnos() {  
    for (Turno turno : turnosRegistrados) {  
        System.out.println(turno);  
    }  
}
```

```
}
```